

Code Explanation

Unit Test Explanation for SFDC Account ETL Module

This test suite is designed to validate the functionality of the SFDC Account ETL module, ensuring that data extraction, transformation, and loading are performed correctly.

Overview of the Code

The provided code is a comprehensive unit test suite for the SFDC Account ETL module. It covers various test cases for individual ETL functions as well as the overall ETL pipeline. The test class, `TestSFDCAccountETL`, contains multiple test methods that verify the correctness of the ETL process.

Step 1: Setting Up the Test Environment

The test environment is set up using the `setUpClass` method, which is a class method that is called before any tests are executed. This method creates a SparkSession and defines a sample data schema and DataFrame.

```
@classmethod
def setUpClass(cls):
    """Set up SparkSession for all test cases"""
    cls.spark = SparkSession.builder
        .appName("SFDC Account ETL Tests")
        .master("local[2]")
        .getOrCreate()

    # Define sample data schema
    cls.sample_schema = StructType([
        StructField("SK_ACCOUNT_ID", DecimalType(15, 0), False),
        StructField("DW_SF_ACCOUNT_ID", DecimalType(15, 0), False),
        StructField("ACC_NAME", StringType(), True),
        StructField("ACC_PARTNER_REGION", StringType(), True),
        StructField("ACC_ACCOUNT_NUMBER", StringType(), True),
        StructField("ACC_PROSPECT_NUMBER", StringType(), True),
        StructField("DW_UPDATE_DT", DateType(), True)
    ])

    # Create sample data
    data = [
        (10000000000001, 20000000000001, "Test Account 1", "NAMR", "AC
C123", "PROS456", datetime(2023, 1, 15)),
        (10000000000002, 20000000000002, "Test Account 2", "NAMR", "AC
C789", "PROS012", datetime(2023, 1, 16)),
        (10000000000003, 20000000000003, "Test Account 3", "EMEA", "AC
C345", "PROS678", datetime(2023, 1, 17)),
        (10000000000004, 20000000000004, "Test Account 4", "NAMR", "AC
C901", "PROS234", datetime(2023, 1, 10))
    ]

    cls.sample_df = cls.spark.createDataFrame(data, cls.sample_schem
a)
```

Step 2: Testing Data Extraction

The `test\_extract\_data` method tests the `extract\_data` function by mocking the SparkSession and

verifying that the correct query is used to extract data.

```
@patch("sfmc_account_integration.src.sfmc_account_etl.spark")
def test_extract_data(self, mock_spark):
    """Test data extraction with filters"""
    # Setup mock
    mock_spark.read.format.return_value.options.return_value.option.return_value.load.return_value = self.sample_df

    # Call function with test date
    result_df = extract_data(mock_spark, "01/15/2023")

    # Verify correct query format was used
    mock_spark.read.format.assert_called_with("jdbc")
    mock_spark.read.format.return_value.options.return_value.option.assert_called_with(
        "query", unittest.mock.ANY)

    # Check the query contains our filters
    query_arg = mock_spark.read.format.return_value.options.return_value.option.call_args[0][1]
    self.assertIn("ACC_PARTNER_REGION = 'NAMR'", query_arg)
    self.assertIn("TRUNC(DW_UPDATE_DT) >= TO_DATE('2023-01-15'", query_arg)
```

Step 3: Testing Data Transformation

The `test\_transform\_data` method tests the `transform\_data` function by applying the transformation to the sample DataFrame and verifying that the resulting DataFrame has the expected columns and data.

```
def test_transform_data(self):
    """Test data transformation logic"""
    # Apply transformation
    transformed_df = transform_data(self.sample_df)

    # Check column renaming
    self.assertIn("ACC_ACCOUNT_DECIMAL", transformed_df.columns)
    self.assertIn("ACC_PROSPECT_DECIMAL", transformed_df.columns)
    self.assertNotIn("ACC_ACCOUNT_NUMBER", transformed_df.columns)
    self.assertNotIn("ACC_PROSPECT_NUMBER", transformed_df.columns)

    # Check data is preserved
    self.assertEqual(transformed_df.count(), self.sample_df.count())
```

Step 4: Testing Data Loading

The `test\_load\_data` method tests the `load\_data` function by mocking the DataFrame and verifying that the data is loaded correctly to the target.

```
@patch("sfmc_account_integration.src.sfmc_account_etl.logger")
def test_load_data(self, mock_logger):
    """Test data loading to target"""
    # Create mock DataFrame with write methods
    mock_df = MagicMock()
    mock_df.write.format.return_value.options.return_value.mode.return_value
```

```

rn_value.save.return_value = None
mock_df.count.return_value = 2

# Call function
target_config = {"url": "jdbc:oracle:test", "dbtable": "TEST_TABLE"}
load_data(mock_df, target_config)

# Verify write was called with correct parameters
mock_df.write.format.assert_called_with("jdbc")
mock_df.write.format.return_value.options.assert_called_with(**target_config)
mock_df.write.format.return_value.options.return_value.mode.assert_called_with("append")
mock_df.write.format.return_value.options.return_value.mode.return_value.save.assert_called_once()

# Verify logging
mock_logger.info.assert_called_with("Successfully loaded 2 rows to target")

```

Step 5: Testing the ETL Pipeline

The `test\_extract\_transform\_load` method tests the `extract\_transform\_load` function by mocking the individual ETL functions and verifying that they are called in the correct order.

```

@patch("sfdc_account_integration.src.sfdc_account_etl.extract_data")
@patch("sfdc_account_integration.src.sfdc_account_etl.transform_data")
@patch("sfdc_account_integration.src.sfdc_account_etl.load_data")
def test_extract_transform_load(self, mock_load, mock_transform, mock_extract):
    """Test the full ETL pipeline"""
    # Setup mocks
    mock_spark = MagicMock()
    mock_extract.return_value = self.sample_df
    mock_transform.return_value = self.sample_df

    # Call function
    extract_transform_load(mock_spark, "01/15/2023")

    # Verify all steps were called in order
    mock_extract.assert_called_once_with(mock_spark, "01/15/2023")
    mock_transform.assert_called_once_with(self.sample_df)
    mock_load.assert_called_once()

```

Step 6: Testing the Source Schema

The `test\_get\_source\_schema` method tests the `get\_source\_schema` function by verifying that the returned schema has the expected fields and data types.

```

def test_get_source_schema(self):
    """Test source schema definition"""
    schema = get_source_schema()

    # Verify schema has expected fields with correct types

```

```
self.assertTrue(isinstance(schema, StructType))

# Check key fields
field_names = [field.name for field in schema.fields]
self.assertIn("SK_ACCOUNT_ID", field_names)
self.assertIn("DW_SF_ACCOUNT_ID", field_names)
self.assertIn("ACC_PARTNER_REGION", field_names)
self.assertIn("DW_UPDATE_DT", field_names)

# Check specific field properties
sk_account_field = next(f for f in schema.fields if f.name == "SK_ACCOUNT_ID")
self.assertFalse(sk_account_field.nullable)
self.assertTrue(isinstance(sk_account_field.dataType, DecimalType))
self.assertEqual(sk_account_field.dataType.precision, 15)
```